



Estado de los componentes y eventos



**Activen las cámaras los que puedan y
pasemos asistencia**

Construir una aplicación en React que modifique el estado de los componentes a partir de eventos e interacciones con el usuario.

{desafío}
latam_

- Unidad 1: Introducción a React.
- Unidad 2: Estados de los componentes y eventos.
- Unidad 3: Renderización dinámica de componentes.
- Unidad 4: Consumo de APIs con React.
- Unidad 5: React Router I
- Unidad 6: Context
- Unidad 7: React Router II
- Unidad 8: JWT



Te encuentras aquí





Inicio

{desafío}
latam_



*/** Crear componentes interactivos, o sea, que modifiquen su estado a partir de eventos e interacciones con el usuario. **/*

**Objetivo de
la unidad**

*/** Instalar la extensión de Google Chrome para inspeccionar elementos y componentes de React. **/*

*/** Modificar un estado en base al clic de un botón. **/*

*/** Modificar un estado en base a la modificación de un input. **/*

Objetivos

Activación de conceptos

Contesta la pregunta correctamente y gana un punto

Instrucciones:

- Se realizará una pregunta, el primero en escribir “YO” por el chat, dará su respuesta al resto de la clase.
- El docente validará la respuesta.
- En caso de que no sea correcta, dará la oportunidad a la segunda persona que dijo “Yo”.
- Cada estudiante podrá participar un máximo de 2 veces.
- Al final, el/la docente indicará el 1º, 2º y 3º lugar.
- Esta actividad no es calificada, es solo una dinámica para recordar los conceptos clave para abordar esta sesión.





Activación de conceptos



¿Cuál es la diferencia entre una función declarativa y una función de expresión?



Activación de conceptos



¿Qué problema hay en el siguiente código?



Activación de conceptos



```
import Producto from './Producto'
```

```
function App() {
```

```
  return (
```

```
    <Producto />
```

```
    <Producto />
```

```
    <Producto />
```

```
  )
```

```
}
```



Activación de conceptos



¿Cuál es la convención al momento de nombrar
nuestros componentes en React?



Activación de conceptos



¿Para qué sirven las props?



Activación de conceptos



¿Qué problema hay en el siguiente código?



Activación de conceptos



Tenemos un Componente llamado **Tienda.jsx** y queremos importarlo en el componente principal **App.jsx**.

¿Qué debemos agregar para que funcione?

```
1  import React from 'react'
2
3  const Tienda = () => {
4    return (
5      <h1>Mi tienda de productos</h1>
6    )
7  }
```



Activación de conceptos



Primer lugar:



Segundo lugar:



Tercer lugar:



Desarrollo

{desafío}
latam_



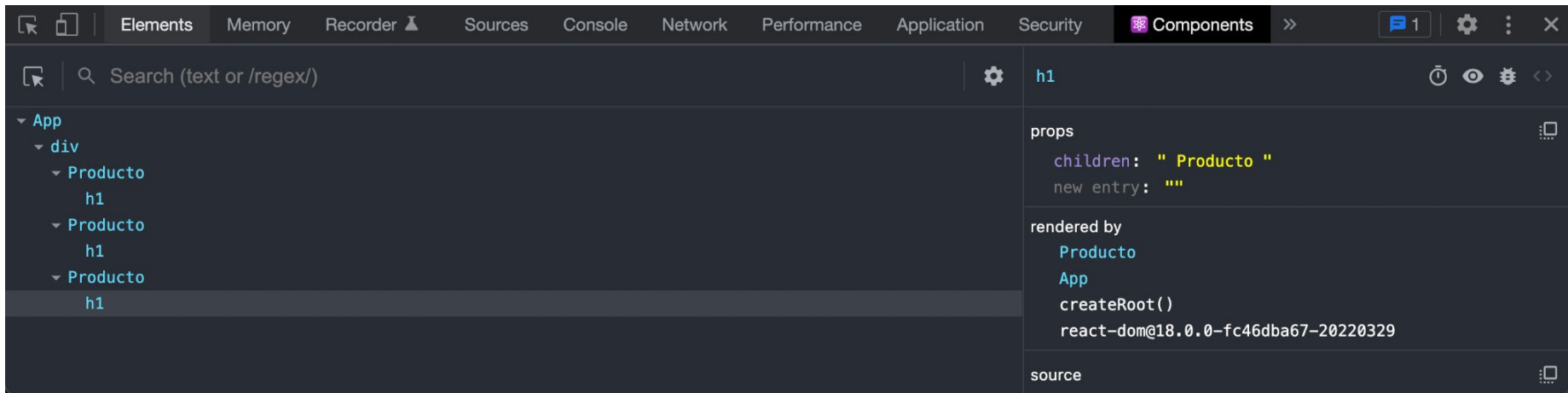
React-developer-tools

- En Chrome tenemos disponible varios plugins para potenciarlo.
- Para trabajar junto a React tenemos una herramienta que nos permitirá depurar nuestras aplicaciones.
- Lo podrás conseguir en el siguiente [enlace](#).



React-developer-tools

Una vez que lo tengamos instalado, revisemos el inspector de elementos del navegador chrome para observar el panel de instalación asociado a React. Si no aparece, reinicia el navegador.



Ejercicio

Una vez instalado nuestro plugin en el navegador realizaremos las siguientes acciones:

- Utilicemos de ejemplo el **Apoyo - Adopta un perrito**.
- Levantemos el servidor para mostrarlo en el navegador.
- Abrir el inspector de elementos del navegador e ir a la pestaña de **Components**.
- Inspeccionamos la herramienta y vamos a identificar los componentes que tenemos creados.

Demostración



Ejercicio

Practiquemos lo aprendido

- Crea un nuevo componente que se llame Descripción.jsx.
- Este componente recibirá un texto mediante props. Para el texto aleatorio puedes utilizar [Lorem Ipsum](#).
- Comprueba el nuevo componente agregado al árbol desde la herramienta de inspección del navegador.

Ejercicio

¡Manos al teclado!



React-developer-tools

Como pudimos observar, esta herramienta de navegador nos sirve para:

1. Depurar nuestras aplicaciones.
2. Tener una visión detallada de nuestro árbol de componentes.
3. Identificar las propiedades que tienen nuestros componentes.
4. Gestionar los estados (Esto último es lo que veremos en esta sesión).

*/** Instalar la extensión de Google Chrome para inspeccionar elementos y componentes  React. **/*

*/** Modificar un estado en base al clic de un botón. **/*

*/** Modificar un estado en base a la modificación de un input. **/*

Objetivos

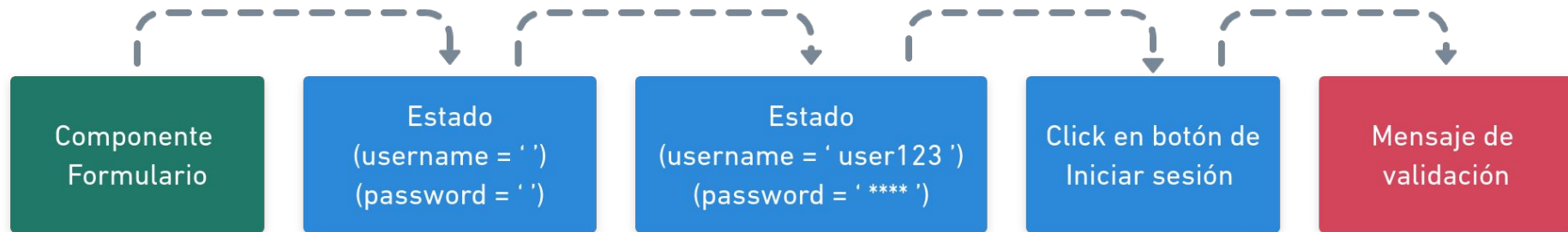
¿Qué es el estado en React?



- Nos permiten almacenar, utilizar y mostrar información actualizada en nuestros componentes.
- El manejo de los estados permiten construir componentes que sean interactivos, y de este modo, mostrar valores o datos a partir de eventos y acciones que realicen los usuarios en el sitio web.
- Para manejar estados en React ocuparemos una función especial llamada useState (usar Estado).

Veamos un ejemplo

Pensemos en el estado como una memoria a corto plazo que maneja cada componente; el estado permitirá que los componentes mantengan esta información, la usen y la actualicen a medida que ocurren cambios en la aplicación.



Ejemplo carrito de compras



El estado comienza con el carrito vacío, luego se agrega el producto A y el estado almacena este valor, luego se elimina el producto A y se agregó el producto el B y C, esto quiere decir que mientras los datos permanezcan en el estado, se pueden realizar operaciones como eliminar o actualizar, finalmente el proceso termina con alguna operación como comprar o vaciar el carrito.

Conozcamos useState()

1. Tenemos que importar useState en el componente donde utilizaremos estados:

```
import { useState } from "react";
```

2. Utilizamos useState y asignamos un valor inicial al estado, obtenemos un valor y una función. La función nos permite actualizar el estado.

```
const [state, setState] = useState(initialState);
```

3. Utilizando la función podemos modificar el valor del estado, ejemplo: setState(newState);

El uso de useState es extraño la primera vez que lo utilizamos, pero su uso siempre sigue los mismos pasos y aprenderemos a manejarlo con práctica.

Nuestra primera componente con useState()

Un contador



Vite + React

- Al crear un proyecto con Vite, este nos genera un contador que usa el estado.

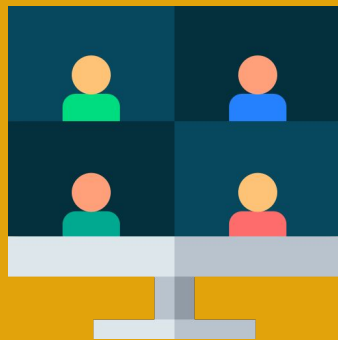
count is 0

Ejercicio

A continuación analizaremos el contador que se genera al crear un proyecto con Vite, el cual utiliza `useState()` para modificar el estado del componente. Para ello deberemos:

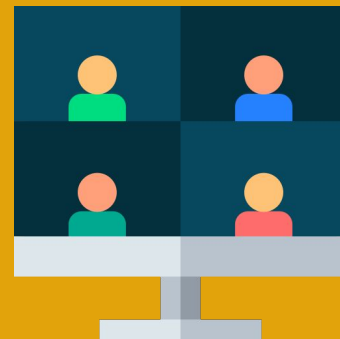
- Crear una nueva aplicación llamada `contador-clicks` usando el comando **`npm create vite`**.
- Instalar las dependencias necesarias con el comando **`npm install`**.
- Inicializamos el servidor con el comando **`npm run dev`**.

Demostración



- En nuestro **App.jsx** observamos que en la primera línea se hace la importación de `useState`, con esto podremos almacenar y modificar su estado.
 - El estado de nuestro contador partirá en 0, esto define al comienzo de la función **App()** antes del **return**.
 - Más abajo encontraremos el botón que usa el estado, al cual se le agrega el evento clic para aumentar su valor en 1 cada vez que se haga esta acción.
- Veamos en react-developer-tools como cambia el estado.

Demostración



Ejercicio

Practiquemos lo aprendido

1. Sobre el proyecto creado agregar un nuevo botón y que incremente el contador de 2 en 2.
2. Verifiquemos que funcione en el navegador y en React-developer-tools

Ejercicio ¡Manos al teclado!



*/** Instalar la extensión de Google Chrome para inspeccionar elementos y componentes de React. **/* 

*/** Modificar un estado en base al clic de un botón. **/* 

*/** Modificar un estado en base a la modificación de un input. **/*

Objetivos

Ejercicio

Realicemos un ejercicio guiado:

1. Crear una nueva aplicación llamada **validacion-input**.
2. Limpiar el código que no utilizaremos dentro del archivo **App.jsx**.
3. Dentro de la carpeta `src`, crearemos una carpeta para nuestros componentes, y dentro de ella, crearemos el componente **MyInput.jsx**.
4. Dentro del componente, agreguemos un input (no es necesario agregar el formulario para este ejercicio).

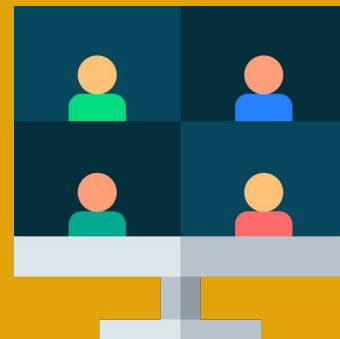
Demostración



Ejercicio

5. Importemos useState.
6. Agreguemos el estado nombre.
7. Utilicemos useState para mapear el estado del input.
8. Agreguemos el evento onChange al input.
9. Mostremos el valor del input en un texto.

Demostración



Ejercicio

Practiquemos lo aprendido

1. Agregar un segundo input.
2. Agregar un nuevo estado llamado color.
3. Mostrar el color en pantalla.
4. Opcional: Utilizar CSS para cambiar el color de fondo al color del input.

Ejercicio ¡Manos al teclado!



*/** Instalar la extensión de Google Chrome para inspeccionar elementos y componentes de React. **/* 

*/** Modificar un estado en base al click de un botón. **/* 

*/** Modificar un estado en base a la modificación de un input. **/* 

Objetivos



Cierre

{desafío}
latam_



- Revisemos el material asincrónico de la unidad.
 - Validación de un formulario con múltiples inputs
 - preventDefault
 - onSubmit()
- Revisemos el desafío de la unidad leyendo la descripción y requerimientos.

¿Qué sigue?

¿Tienen alguna duda respecto al Desafío?



*Academia de
talentos digitales*

www.desafiolatam.com



/DesafioLatam



/DesafioLatam



/DesafioLatam



/DesafioLatam